Este documento está protegido por la Ley de Propiedad Intelectual (Real Decreto Ley 1/1996 de 12 de abril). Queda expresamente prohibido su uso o distribución sin autorización del autor.

UNIVERSIDAD DE GRANADA

Curso 2025 -26

Inteligencia Artificial

2º Grado en Ingeniería Informática

Guion de la Práctica 1: Introducción a la IA Generativa

Ejercicio 2: Generación de Código con IA Generativa

1. Objetivo 2
2. Requisitos para realizar el ejercicio 2
3. Realización del ejercicio 2
4. Entrega del ejercicio 2
5. Introducción 3
6. Ingeniería de prompts 4
7. Representación de estados del mundo y del problema 6
8. Representación de operadores (dinámica del mundo) 7
9. Generación del espacio de estados. 8
10. Búsqueda de la solución del problema 8
11. Siguientes pasos 9

DECSAI

Departamento de Ciencias de la Computación e Inteligencia Artificial

Generación de Código para resolver el 8-puzzle con IA Generativa

1. Objetivo

Usar IA generativa para desarrollar un programa en Python que resuelva el problema del 8-puzzle, teniendo en cuenta las consideraciones comentadas en el seminario. El alumno definirá prompts, completando las plantillas que se proponen más abajo, para la generación de código Python, con el objeto de comprender cuales son los componentes que forman la descripción de un problema en Inteligencia Artificial. Podrá usarse cualquiera de los chatbots de IA Generativa que hemos visto en clase (Gemini3, ChatGPT Free, DeepSeek, o cualquier otro que conozca el alumno).

2. Requisitos para realizar el ejercicio

Para realizar este ejercicio es necesario habilitar una cuenta en cada uno de los asistentes conversacionales de IA Generativa, o chatbots, que hemos visto en el seminario.

Gemini: para acceder a Gemini, https://gemini.google.com/, es suficiente con que uséis la cuenta <usuario>go.ugr.es de la UGR.

DeepSeek: para DeepSeek, https://www.deepseek.com/, podéis crear de forma fácil una cuenta con vuestra cuenta <usuario>go.ugr.es de la UGR, o cualquier otra cuenta de correo de Google que creéis aparte.

ChatGPT(free): para ChatGPT(free), https://chatgpt.com/, podéis hacer lo mismo que para DeepSeek.

Por otro lado, es muy común la generación de código con IA Generativa basada en Python, por tanto, es recomendable tener instalado Python.

- Cómo instalar Python: https://chatgpt.com/share/67b461bf-dd1c-8006-809b-c19eea4846bd

No es necesario conocer Python, la IA Generativa generará el código, pero al menos es necesario saber cómo ejecutar un script en Python para comprobar su salida.

- Cómo ejecutar un programa Python en VS Code https://chatgpt.com/share/67b47571-a2a4-8006-b8f6-932d90afe29a
- Cómo ejecutar un programa Python desde la Shell https://chatgpt.com/share/67b4761d-5988-8006-b6fb-74ed2599dfe2

3. Realización del ejercicio

La realización del ejercicio consiste en llevar a cabo las tareas que se indican más adelante sobre escritura de prompts para resolver el 8-puzzle y responder a las preguntas que se encuentran en un documento aparte llamado *Cuestionario Ejercicio 2*, disponible en el espacio de prácticas de la asignatura en Prado.

4. Entrega del ejercicio

Para entregar el ejercicio:

1. Rellenar el cuestionario que se ha indicado arriba y usar el enlace para la entrega del ejercicio 2 habilitado en Prado.
2. Subir en un fichero zip los archivos generados durante el chat con la IA Generativa: estado.py, acciones.py, espacio.py y resolver-BFS.py que se irán creando y guardando por el alumno conforme se desarrolle la tarea.

5. Introducción

En este ejercicio se usará la IA generativa para ayudarnos a escribir un programa para resolver el problema del 8-puzzle. La IA Generativa es una herramienta muy potente que puede ayudarnos a desarrollar software y a comprender mejor los conceptos de Inteligencia Artificial. Sin embargo, es importante recordar que no "razona" mediante métodos sistemáticos y formales (como la IA simbólica), sino que genera respuestas basándose en patrones y probabilidades aprendidas a partir de grandes cantidades de datos. Por ello, el código o las explicaciones que produce pueden contener errores o imprecisiones.

Su uso debe ir siempre acompañado de análisis crítico, comprobación y validación por parte del estudiante. La IA generativa debe emplearse de forma que el programa final ofrezca garantías razonables de corrección y mantenga una estructura modular que facilite su comprensión, mantenimiento y mejora posterior. Solo así el uso de estas herramientas supone un ahorro real de tiempo y no una fuente adicional de errores.

Para fomentar buenas prácticas de diseño y facilitar el proceso de verificación, se recomienda organizar el código en distintos ficheros ".py", separando cada componente fundamental del problema del 8-puzzle (representación, operadores, transición, búsqueda, etc.), tal como se explica a continuación.

Un problema en IA se define a partir de los siguientes componentes (se explicará en teoría en más detalle en el Tema 3):

- **Representación de estados y representación del problema.**
  - **Representación de un estado**: Definición formal de cómo se describe cualquier configuración posible del sistema (por ejemplo, cómo se codifica un estado del 8-puzzle).
  - **Estado inicial**: Configuración desde la que comienza el proceso de búsqueda.
  - **Estado objetivo:** Configuración que se desea alcanzar.
  - **Condición de objetivo**: Criterio que permite determinar si un estado dado coincide con el estado objetivo.
  - **Representación del problema:** Definición formal del problema como el par formado por el estado inicial y el estado objetivo, junto con el conjunto de operadores o acciones que permiten transformar estados.

    La representación de estados y problema se escribirá mediante la ayuda de la IA Generativa en un fichero llamado ***estado.py***

- **Representación de acciones**. La representación de acciones define la **dinámica del mundo**, es decir, cómo pueden transformarse los estados. También se conoce como representación de operadores. Incluye los siguientes elementos:
  - **Nombre simbólico del operador:** Identificador que representa cada acción posible (por ejemplo, U, D, L, R).
  - **Precondiciones:** Condiciones que deben cumplirse en un estado para que el operador pueda aplicarse.
  - **Efectos (función de transición):** Especificación formal del nuevo estado que resulta de aplicar el operador a un estado en el que se cumplen sus precondiciones.

    La representación de acciones se escribirá en el fichero ***acciones.py***

- **Representación del espacio de estados:** La representación del espacio de estados define el conjunto de todos los estados alcanzables a partir del estado inicial mediante la aplicación de operadores. Incluye:
    - **Función de generación de sucesores:** Definición de una función que, dado un estado, devuelve el conjunto (o lista) de estados sucesores que pueden obtenerse aplicando los operadores aplicables en dicho estado.

    La representación del espacio de estados se escribirá en el fichero ***espacio.py***

- **Solución del problema**:
    - La resolución del problema se realiza mediante un **algoritmo de búsqueda**, que explora el espacio de estados a partir del estado inicial. El proceso consiste en generar estados sucesores mediante la aplicación de operadores, expandir iterativamente nuevos estados y detenerse cuando se alcanza un estado que satisface la condición de objetivo.
    - La **solución del problema** se representa como una secuencia ordenada de operadores que, aplicados sucesivamente al estado inicial, permiten alcanzar el estado objetivo.

    El algoritmo de búsqueda y sus funciones auxiliares se escribirán en el fichero ***resolver.py***

No es necesario conocer Python, pero sí cómo ejecutar un script o programa Python. Puedes hacer uso de la IA Generativa para que te explique **como ejecutar un script o programa Python** dependiendo del entorno (Linux/Windows con o sin VS Code).

6. Ingeniería de prompts

La **Ingeniería de prompts** es el conjunto de técnicas y buenas prácticas orientadas a diseñar y formular instrucciones claras, precisas y estructuradas para interactuar eficazmente con modelos de Inteligencia Artificial generativa. Su objetivo es obtener respuestas lo más correctas, útiles y alineadas posible con la tarea deseada, reduciendo ambigüedades y errores. Sirve para mejorar la calidad del contenido generado, guiar el comportamiento del modelo y aprovechar mejor sus capacidades en tareas como programación, análisis de información o generación de texto. En la actualidad, la Ingeniería de prompts es especialmente relevante porque se ha convertido en una competencia clave para utilizar de manera crítica y eficiente las herramientas de IA generativa en contextos académicos y profesionales.

A continuación, se presentan prompts que debes usar para indicar a la IA Generativa como desarrollar el programa Python. Es importante seguir buenas prácticas en la escritura de prompts. Las siguientes reglas pueden guiar en el proceso de "prompt engineering".

REGLAS ESENCIALES PARA ESCRIBIR PROMPTS DE GENERACIÓN DE CÓDIGO

1. Define exactamente qué debe hacer el código. Describe la tarea de forma concreta y sin ambigüedades. Evita peticiones generales como "haz un programa completo". Especifica claramente el objetivo funcional.
2. Especifica claramente las entradas y las salidas. Indica el tipo de datos de entrada, el formato esperado y la estructura de datos que debe devolver la función o programa. Cuanto más preciso sea el formato, mejor será el resultado.

3. Limita el alcance del código generado. Indica explícitamente lo que no debe hacer el código. Por ejemplo: "No incluyas el algoritmo de búsqueda", "No modifiques el estado original", "No generes código adicional". Esto evita que la IA añada partes no solicitadas.
4. Exige estructura y modularidad. Especifica el nombre de las funciones, la organización en ficheros y la necesidad de usar funciones auxiliares si procede. Esto ayuda a mantener el código organizado y coherente.
5. Verifica y cuestiona el resultado. Nunca aceptes el código sin comprobarlo. Es necesario ejecutarlo, probar casos normales y casos límite, y verificar que cumple exactamente las precondiciones y el comportamiento esperado.

En las siguientes secciones se van a presentar varios recuadros, cada uno contiene una plantilla de prompt para generar código Python. El alumno debe completar las partes del prompt delimitadas con **< >. Una vez completado el prompt,** se proporcionará como entrada al chatbot elegido para generar las funciones Python, crear su archivo correspondiente y copiar el código en el mismo. Los prompts contienen texto explicativo para que la IA Generativa produzca código contemplando las dependencias entre archivos.

7. Representación de estados del mundo y del problema.

Representación de estados del mundo

Actúa como un asistente experto en programación en Python.

Definimos un estado del 8-puzzle como *<completa indicando la estructura de datos para representar un estado>*.

Genera una función "representación_estado" que reciba como entrada una cadena de 9 números enteros sin repeticiones en el intervalo [0,8] que representen las fichas del problema del 8-puzzle, considerando que el valor 0 representa la casilla vacía. Por ejemplo "123456780". La función debe construir una representación interna del estado del mundo del 8-puzzle mediante la estructura de datos previamente definida y devolverla.

Genera una función en Python para mostrar por pantalla una representación visual de un estado del 8-puzzle clara y amigable para un humano, donde la casilla vacía se represente de forma distinguible (por ejemplo, con un espacio en blanco o un símbolo).

Definimos un problema del 8-puzzle como un par de estados, inicial y objetivo, cada uno representados según la estructura definida previamente. Define una clase python para representar un problema del 8-puzzle

Genera un método para la clase Problema llamado "representacion_problema" *<especificar entrada y formato de la misma, sus restricciones, indicando un ejemplo, qué debe hacer la función y qué devuelve>*.

Genera una función en Python para mostrar por pantalla una representación visual de un problema del 8-puzzle clara y amigable para un humano, donde la casilla vacía se represente de forma distinguible (por ejemplo, con un espacio en blanco o un símbolo).

Definimos que un estado dado, con la representación previamente definida, **es estado objetivo** cuando <describir las condiciones que debe cumplir un estado para considerarse estado objetivo>.

Genera una función como método de la clase Problema que reciba como entrada un estado cualquiera del 8-puzzle y devuelva si dicho estado es estado objetivo.

Todas estas funciones deben ir en un mismo fichero denominado **estado.py**

Genera las funciones que acabos de indicar. Incluir comentarios breves que expliquen el propósito de cada parte del código. Asume que la entrada es válida y céntrate exclusivamente en la representación del estado y su visualización.

Verificación: el alumno debe verificar que las funciones escritas por la IA Generativa son correctas antes de pasar al siguiente paso.

8. Representación de operadores, operadores aplicables a un estado y función de transición (dinámica del mundo)

Representación de operadores, operadores aplicables a un estado y función de transición (dinámica del mundo)

Definimos que los operadores aplicables en el mundo del 8 puzzle se **representan** como <identificar los nombres de los operadores y describir qué hace cada operador>

Definición de las precondiciones: condiciones de aplicabilidad de un operador en un estado dado.

<explicar en lenguaje natural, pero preciso, las condiciones de aplicabilidad de cada operador>

Genera una función python para definir las condiciones de aplicabilidad de cada operador. Cada función se llamará EsAplicable<operador>, <describe la entrada, estructura de la misma y la salida de estas funciones. Describe qué deben hacer estas funciones>.

Definición de los efectos (función de transición): definición del estado resultante de la aplicación de un operador/acción a un estado dado.

<Describe por cada operador una función que defina sus efectos, su entrada y salida y qué debe calcular>

Genera la función de transición, que define la dinámica del mundo. Es una función que se llamará AplicaOperador y<describe la entrada, su formato, y la salida, y su formato, y qué debe calcular la función>.

La **función de transición** hará uso de funciones que tienen que generarse para aplicar cada uno de los operadores de forma modular <describe qué es necesario para que la función de transición haga uso, siguiendo la filosofía de programación modular, de las funciones previamente definidas para cada operador>

Todas estas funciones deben estar en el fichero acciones.py que debe importar el fichero estado.py creado previamente.

.

Verificación: el alumno debe verificar que las funciones escritas por la IA Generativa son correctas antes de pasar al siguiente paso.

9. Generación del espacio de estados.

El espacio de estados está definido como un grafo que contiene todos los nodos que pueden generarse a partir de la aplicación de los operadores al estado inicial y sus sucesores (se explicará en detalle en el Tema 3 en teoría).

Generación del espacio de estados

Genera una función que se llame EstadosSucesores para un estado dado <explica que entrada tiene que tener, su formato y qué salida, así como su formato/estructura, además de qué tiene que calcular la función>.

Esta función estará en el fichero espacio.py que importará el fichero estado.py y el fichero acciones.py.

Verificación: el alumno debe verificar que las funciones escritas por la IA Generativa son correctas antes de pasar al siguiente paso.

10. Búsqueda de la solución del problema

Solución del problema:

La solución al problema se lleva a cabo mediante un proceso de búsqueda que consiste en un proceso que parte del estado inicial, genera estados sucesores, aplica de forma iterativa la generación de sucesores para cada estado previamente generado y se detiene cuando uno de los sucesores cumple las condiciones de estado objetivo.

La representación de la solución es una secuencia de operadores que aplicados al estado inicial nos permiten alcanzar el estado objetivo.

Genera una función que Soluciona8Puzzle q<describir la entrada y salida de la función, con precisión respecto a su formato y estructura>

El proceso de búsqueda estará basado en el algoritmo de búsqueda en anchura y deberá hacer uso de todas las funciones previas que se han definido en los ficheros estado.py, acciones.py y espacio.py.

Genera una función para resolver el 8-puzzle siguiendo las instrucciones previas y hazlo de forma modular para que la estructura de datos usada para representar el espacio de búsqueda interno de la función pueda ser modificada fácilmente y cambiar, por ejemplo, el comportamiento a búsqueda en profundidad.

La función y posibles funciones auxiliares que necesiten generarse deben estar en el fichero resolver_bfs.py que incluye los ficheros estado.py, acciones.py y espacio.py.

Verificación: el alumno debe verificar que las soluciones generadas por el algoritmo de búsqueda son correctas, es decir, que la secuencia de pasos generada permite ir desde el estado inicial hasta el objetivo.

11. Siguientes pasos

Una vez realizadas todas las tareas arriba indicadas, pasar a responder las cuestiones planteadas en el cuestionario sobre el ejercicio 2.